DATA MINING PROJECT

DEVELOPMENT OF A DIABETES RISK PREDICTION MODEL USING SUPERVISED MACHINE LEARNING

PREPARED BY

ANN JEPCHUMBA ROTICH

Student ID: sjepro2412

I, Ann Jepchumba Rotich, confirm that I have reviewed this project report and take responsibility for the work presented. Sources consulted in preparing the report are acknowledged in the references. The model is intended for academic study and is not a medical diagnostic device.

Name: Ann Jepchumba Rotich
Student ID: SJEPRo2412

ABSTRACT

Diabetes is a chronic health condition that can lead to serious complications when it is not identified and managed appropriately. This project proposes the development of a supervised machine learning model that classifies patient records into diabetes-positive and diabetes-negative categories using routinely recorded health measurements. The project uses the Pima Indians Diabetes dataset, which contains 768 records, eight predictor variables and one binary outcome. The proposed workflow includes data understanding, cleaning, exploratory analysis, stratified training and testing, model training, and performance evaluation. Logistic Regression, Decision Tree and Random Forest classifiers are compared, with Random Forest selected as the principal candidate because it combines multiple decision trees and can model non-linear relationships. Missing or implausible zero values in selected clinical measurements are treated as missing and imputed using training-set medians within a machine-learning pipeline. The accompanying Python implementation calculates accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices for the three classifiers. The model is an academic screening-support prototype and is not a substitute for professional diagnosis.

Keywords: Data mining, diabetes, supervised learning, classification, Random Forest, Logistic Regression, Decision Tree.

TABLE OF CONTENTS

• Chapter One: Introduction

• 1.1 Background of the Study

• 1.2 Problem Statement

• 1.3 Aim and Objectives

• 1.4 Research Questions

• 1.5 Scope and Significance

• Chapter Two: Literature and Technical Background

• 2.1 Data Mining and Supervised Learning

• 2.2 Classification Algorithms

• 2.3 Diabetes Dataset and Its Limitations

• Chapter Three: Methodology and System Design

• 3.1 Research Design and Dataset

• 3.2 Variables and Data Dictionary

• 3.3 Data Preparation

• 3.4 Model Development

• 3.5 Evaluation and Validation

• 3.6 Ethical and Safety Considerations

• Chapter Four: Implementation Plan and Results Framework

• 4.1 Tools and Environment

• 4.2 Implementation Steps

• 4.3 How to Run the Python Program

• 4.4 Results and Discussion

• Chapter Five: Summary and Conclusion

• References

• Appendix A: Python Source Code

• Appendix B: Sample Prediction Input

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

Diabetes is a chronic disease in which the body does not produce enough insulin or cannot use the insulin it produces effectively. Persistently raised blood glucose can damage blood vessels and nerves and may contribute to complications such as kidney disease, vision loss, cardiovascular disease and lower-limb problems. The World Health Organization reports that the number of people living with diabetes increased from about 200 million in 1990 to 830 million in 2022, with the burden rising more rapidly in low- and middle-income countries. The International Diabetes Federation estimated that 589 million adults aged 20–79 were living with diabetes in 2024 and projected that this could rise to 853 million by 2050. These figures illustrate why prevention, screening and timely clinical assessment remain important public-health priorities.

Healthcare services generate and use data such as glucose measurements, body mass index, blood pressure, age and family-history indicators. Data mining can examine such information to discover patterns that may be difficult to identify through manual inspection alone. Supervised machine learning is particularly suitable when historical examples include known outcome labels. A model can learn relationships between predictor variables and the recorded outcome, then estimate the likely class for a new record. In this project, the intended classes are diabetes-positive and diabetes-negative.

The proposed project develops a binary classification model using the Pima Indians Diabetes dataset. The dataset is widely used in introductory predictive-modelling work and contains eight numeric health-related predictors. The project compares Logistic Regression, Decision Tree and Random Forest. It uses careful preprocessing and validation so that the evaluation is performed on records that were not used to train the model. The resulting prototype demonstrates how data mining can support preliminary risk-screening research, while recognising that a statistical prediction is not a medical diagnosis.

1.2 Problem Statement

Diabetes may remain undetected for a period of time, and delayed identification can postpone clinical advice and management. Screening decisions require consideration of multiple measurements and risk indicators. A manual review of many patient records can be time-consuming, and simple single-factor rules may fail to capture interactions among variables such as age, glucose concentration, body mass index and family-history information. There is therefore a need to investigate whether a supervised data-mining model can use available health measurements to classify records consistently and provide an additional signal for further assessment.

This project addresses the problem by building and evaluating a binary classification model trained on labelled patient records. The model will learn from the predictor variables and the recorded diabetes outcome. It will compare several algorithms and evaluate their performance using measures that reveal both correct predictions and missed positive cases. The model is not intended to replace blood tests, clinical judgement or professional diagnosis; it is a learning prototype that demonstrates a possible decision-support approach.

1.3 Aim and Objectives

Main aim: To develop and evaluate a supervised machine learning model for classifying diabetes status from selected health-related attributes.

• To obtain and understand a publicly available labelled diabetes dataset.

• To inspect, clean and preprocess the data, including the treatment of implausible zero values.

• To train Logistic Regression, Decision Tree and Random Forest classification algorithms.

• To compare the algorithms using accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC.

• To identify the most suitable model for the dataset and document its limitations.

1.4 Research Questions

• Can health-related attributes in the selected dataset be used to classify diabetes status?

• How do Logistic Regression, Decision Tree and Random Forest compare on the same test data?

• Which evaluation measures provide the most useful picture of model performance, particularly for identifying positive cases?

1.5 Scope and Significance

The project is limited to a binary classification experiment using the Pima Indians Diabetes dataset. It does not recruit participants, collect new clinical records or deploy the model in a hospital. The model is significant as a practical demonstration of the data-mining process: acquiring data, cleaning it, selecting predictors, training algorithms, validating predictions and interpreting performance. The workflow can help students understand how supervised learning may contribute to healthcare analytics.

The dataset has a narrow population profile: the records represent women aged at least 21 years from the Pima community. Consequently, results cannot automatically be generalised to men, children, other ethnic groups or the wider Kenyan population. Any future local application would require suitable, ethically collected and representative Kenyan data, external validation and clinical oversight.

CHAPTER TWO: LITERATURE AND TECHNICAL BACKGROUND

2.1 Data Mining and Supervised Learning

Data mining is the process of extracting useful patterns, relationships or predictive information from data. A typical predictive data-mining workflow includes defining the problem, collecting data, understanding variables, preparing data, building a model, evaluating results and communicating findings. Supervised learning is a machine-learning approach in which the training dataset contains input features and a known target label. The algorithm learns a mapping from the features to the target and applies that learned mapping to unseen records.

Classification is a supervised-learning task in which the target is a category rather than a continuous quantity. This project is a binary classification problem because the target has two possible values: 0 (recorded diabetes-negative) and 1 (recorded diabetes-positive).

2.2 Classification Algorithms

Logistic Regression

Logistic Regression estimates the probability that an observation belongs to a class. It is a useful baseline for binary classification and provides a relatively interpretable relationship between input variables and the predicted probability. It generally models a linear relationship between the predictors and the log-odds of the positive class. Feature scaling is included in the implementation pipeline for this algorithm.

Decision Tree

A Decision Tree repeatedly splits the data using feature-based rules. It can model non-linear relationships and is relatively easy to explain. However, a tree that grows too deeply may fit noise in the training data and perform less well on new observations. Limiting tree depth and minimum leaf size can help control complexity.

Random Forest

Random Forest is an ensemble method that builds multiple decision trees using bootstrapped samples and random subsets of features. Their predictions are combined to produce a final class or probability. Averaging across trees can improve predictive stability and reduce the variance of an individual tree. Random Forest is selected as the principal candidate in this project because health measurements may have non-linear relationships and interactions. Its performance must nevertheless be measured on held-out data rather than assumed.

2.3 Diabetes Dataset and Its Limitations

The Pima Indians Diabetes dataset is commonly described as containing 768 observations, eight numeric input attributes and one binary class label. The variables include number of pregnancies, two-hour plasma glucose concentration, diastolic blood pressure, triceps skinfold thickness, two-hour serum insulin, body mass index, diabetes pedigree function and age. The commonly distributed version contains 500 negative and 268 positive records. The dataset documentation notes that the subjects are women aged 21 years or older from the Pima Indian population.

A key data-quality issue is that zero appears in some measurement columns where zero is not a plausible clinical measurement for an adult, including glucose, blood pressure, skin thickness, insulin and BMI. In the project implementation, these zeros are treated as missing values for those columns and imputed using medians calculated from the training data. This is a modelling assumption and must be stated because zero values may have different meanings in different datasets. The data also has class imbalance and a limited population profile. These limitations are considered when selecting metrics and interpreting results.

CHAPTER THREE: METHODOLOGY AND SYSTEM DESIGN

3.1 Research Design and Dataset

The project uses a quantitative, experimental machine-learning design. A publicly available secondary dataset is used; no personal information is collected directly from individuals. The selected dataset is the Pima Indians Diabetes dataset, with 768 patient records, eight predictors and one outcome variable. The project code downloads a commonly used CSV mirror of the dataset and assigns the documented column names. The dataset source and access route are listed in the references and in the source code.

The target variable is Outcome, where 0 represents a negative diabetes test in the dataset and 1 represents a positive diabetes test. The independent variables are the eight health-related attributes. The dependent variable is the recorded diabetes outcome.

3.2 Variables and Data Dictionary

Variable	Description	Role
Pregnancies	Number of times pregnant	Predictor
Glucose	Two-hour plasma glucose concentration	Predictor
BloodPressure	Diastolic blood pressure (mm Hg)	Predictor
SkinThickness	Triceps skinfold thickness (mm)	Predictor
Insulin	Two-hour serum insulin	Predictor
BMI	Body mass index	Predictor
DiabetesPedigreeFunction	Family-history-related diabetes pedigree score	Predictor
Age	Age in years	Predictor

Variable	Description	Role
Outcome	0 = negative; 1 = positive	Target

3.3 Data Preparation

• **Data acquisition:** download the CSV dataset and attach the documented column names.

• **Initial inspection:** inspect dimensions, data types, duplicate rows, class distribution, summary statistics and missing values.

• **Data cleaning:** remove exact duplicate records if present; replace zero values with missing values in Glucose, BloodPressure, SkinThickness, Insulin and BMI because such zeros are treated as implausible measurement entries in this dataset.

• **Imputation:** use median imputation for missing numeric measurements. The imputer is fitted only on training data through a pipeline to avoid information leakage.

• **Data splitting:** use an 80% training and 20% testing split with stratification so that the class proportions are approximately maintained.

• **Scaling:** standardise numeric features for Logistic Regression. Decision Tree and Random Forest do not require standardisation, but the same cleaned split is used for fair comparison.

3.4 Model Development

Three supervised classifiers will be trained: Logistic Regression, Decision Tree and Random Forest. Each model will be placed in a scikit-learn pipeline that performs median imputation before classification. Logistic Regression will additionally standardise numeric features. The Random Forest model will use a fixed random seed to support reproducibility and class weighting to help account for unequal class frequencies. The training data will be used for model fitting and cross-validation; the test data will be reserved for final evaluation.

The model-development sequence is: load the dataset; inspect and clean it; separate features from the target; create a stratified train/test split; define preprocessing and model pipelines; fit each classifier; calculate cross-validation scores on training data; evaluate each fitted pipeline on the untouched test data; compare results; and save the selected model with joblib. The final choice should consider recall and F1-score as well as accuracy because failing to identify a positive case may be important in a screening-support context.

3.5 Evaluation and Validation

• **Accuracy:** proportion of all test predictions that are correct.

• **Precision:** proportion of predicted positive cases that are truly positive in the labelled test data.

• **Recall (sensitivity):** proportion of actual positive cases correctly identified.

• **F1-score:** harmonic mean of precision and recall.

• **Confusion matrix:** counts of true negatives, false positives, false negatives and true positives.

• **ROC-AUC:** measures ranking discrimination across classification thresholds, where available.

• **Stratified cross-validation:** assesses performance stability across multiple folds while preserving class proportions.

The test set is not used to fit preprocessing steps or model parameters. This separation reduces optimistic evaluation caused by data leakage. If hyperparameter tuning is conducted, it should occur within

cross-validation on the training set only. The final test set should be evaluated once after model selection.

3.6 Ethical and Safety Considerations

The dataset is publicly available for educational analysis and the project does not attempt to identify individual patients. The report avoids making claims that a predicted class is a diagnosis. A false negative could create inappropriate reassurance, while a false positive could cause unnecessary concern; therefore, any real-world use would require clinical validation, governance, privacy safeguards and appropriate professional oversight. The population limitations of the dataset must be made clear, especially before considering application in Kenya or any population not represented in the original records.

CHAPTER FOUR: IMPLEMENTATION PLAN AND RESULTS FRAMEWORK

4.1 Tools and Environment

• Programming language: Python 3.

• Development environment: Jupyter Notebook, Google Colab or a Python IDE.

• Libraries: pandas, NumPy, scikit-learn, matplotlib, seaborn and joblib.

• Input: the Pima Indians Diabetes CSV dataset.

• Output: trained classification pipelines, evaluation metrics, confusion matrices and a saved model file.

4.2 Implementation Steps

• Install the required Python libraries.

• Download and load the CSV file.

• Check dataset shape, data types, missing values, duplicates and target distribution.

• Replace implausible zeros in selected measurement columns with missing values.

• Separate the eight predictor variables from the Outcome target.

• Split data into 80% training and 20% testing sets using stratification.

• Fit Logistic Regression, Decision Tree and Random Forest pipelines.

• Compare stratified cross-validation results and held-out test metrics.

• Save the selected pipeline and report the final test metrics.

4.3 How to Run the Python Program

Copy the Python source code in Appendix A into a file named diabetes_model.py or a Jupyter Notebook. Install the required packages using the command below. The script downloads the CSV file from the public repository URL, so an internet connection is required for the first run.

```
pip install pandas numpy scikit-learn matplotlib seaborn joblib
```

Run the script with: **python diabetes_model.py**. It prints the model comparison, classification reports, confusion matrices and ROC-AUC where available. It also saves the best-performing model pipeline as **diabetes_risk_model.joblib**. The exact output depends on the downloaded dataset version and installed library versions.

4.4 RESULTS AND DISCUSSION

The performance of Logistic Regression, Decision Tree and Random Forest is considered against a published comparative experiment that used the same Pima Indians Diabetes dataset. Paharia (2025) reports results for all three algorithms on a dataset of 768 records and eight clinical predictors, using five-fold cross-validation. The reported scores are presented below as an external benchmark to help interpret the algorithms selected for this project. They are not represented as measurements produced by the Python implementation accompanying this report.

Classifier	Accuracy	Precision	Recall	F1-score
Logistic Regression	76.3%	72.5%	74.1%	73.3%
Decision Tree	78.6%	76.4%	75.9%	76.1%
Random Forest	82.5%	79.8%	81.0%	80.4%

Source: Paharia (2025), *A Machine Learning Approach to Predicting Diabetes Using Healthcare Data*. The study reports 76.3% accuracy, 72.5% precision, 74.1% recall and 73.3% F1-score for Logistic Regression; 78.6%, 76.4%, 75.9% and 76.1% respectively for Decision Tree; and 82.5%, 79.8%, 81.0% and 80.4% respectively for Random Forest.

The benchmark results show that Random Forest achieved the strongest overall performance among the three algorithms. Its accuracy was 6.2 percentage points higher than Logistic Regression and 3.9 points higher than Decision Tree. It also recorded the highest recall and F1-score, suggesting a better balance between identifying positive cases and limiting incorrect positive predictions in that comparative experiment. These findings support the choice of Random Forest as the main candidate for this project. However, scores can change with data cleaning, train-test partitioning, model settings and validation procedures, so the published figures should not be transferred directly to the project model.

The accompanying Python implementation defines the project-specific evaluation procedure: it cleans the dataset, uses a stratified 80:20 training and testing split, applies five-fold cross-validation to the training data, fits the three classifiers and calculates accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices. The actual output of that implementation is not available in this report; therefore, its own test-set scores and best-performing classifier cannot be independently confirmed here. The objectives of defining the classification problem, preparing the data workflow and providing reproducible model code have been addressed. The objective of empirically validating the developed model has only been partially addressed until the program is executed and its generated metrics are recorded. No clinical effectiveness is claimed.

Benchmark reference: Paharia, A. (2025, April 18). *A Machine Learning Approach to Predicting Diabetes Using Healthcare Data*. https://d197for5662m48.cloudfront.net/documents/publicationstatus/254522/preprint_pdf/1e1e925a9431c63e339901e569bdbc9f.pdf

1. International Diabetes Federation. (2025). IDF Diabetes Atlas (11th ed.). https://diabetesatlas.org/

CHAPTER FIVE: SUMMARY AND CONCLUSION

This project examined the use of supervised machine learning to classify diabetes status from health-related measurements. Using the Pima Indians Diabetes dataset, it established a binary classification workflow based on eight predictor variables and a diabetes outcome label. The project described the data-cleaning and preprocessing steps, including treating implausible zero measurements as missing values, median imputation, a stratified training and testing split, and a comparison of Logistic Regression, Decision Tree and Random Forest. The accompanying Python source code provides a reproducible pipeline for training the classifiers and evaluating their predictions. A published comparison using the same dataset found that Random Forest achieved the highest reported accuracy, precision, recall and F1-score among the three methods considered; however, the project-specific scores must be generated by executing the supplied implementation before its predictive performance can be confirmed. The work therefore meets its design and implementation aims, while the final empirical validation remains an important part of completing the model assessment.

Recommendations for Future Improvement

Future work should run the model on the full dataset and report the actual held-out test results, tune model parameters through cross-validation on the training data, and evaluate the final model on an independent dataset. The dataset used in this project represents a specific population, so a Kenyan application would require appropriately collected and consented local data. Further improvement could include probability calibration, threshold selection that prioritises clinically appropriate recall, fairness checks across relevant population groups, and review by qualified health professionals. The system should remain a research prototype unless it has undergone independent clinical, ethical and technical validation.

GitHub Repository Link

The project source code and supporting files are prepared in the accompanying GitHub-ready archive. The public repository address should be recorded below after the files have been uploaded to GitHub:

GitHub repository URL: _____

The repository should include the Python source file, dependency list, README instructions and final project report.

2. World Health Organization. (2026, November 3). *Diabetes*. https://www.who.int/news-room/fact-sheets/detail/diabetes

3. KEEL. (n.d.). *Pima Indians Diabetes dataset*. https://sci2s.ugr.es/keel/dataset.php?cod=21

4. OpenML dataset documentation. (n.d.). *Pima Indians Diabetes dataset description*. https://github.com/datasets/openml-datasets/blob/main/data/diabetes/README.md

5. scikit-learn developers. (n.d.). *RandomForestClassifier documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

6. scikit-learn developers. (n.d.). *train_test_split documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

7. Brownlee, J. (2018). *Deep Learning with Python*. Machine Learning Mastery. Dataset information and original public CSV reference: https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv

APPENDIX A: PYTHON SOURCE CODE

The following script is a reproducible implementation of the proposed model. It uses pipelines so that imputation and scaling are fitted only on training folds. It compares three classifiers and saves the best model according to test-set F1-score. For a health-screening context, inspect recall and false negatives before deciding whether the model is appropriate even for further research.

```
"""
Diabetes Risk Prediction - Data Mining Project
Author: Ann Jepchumba Rotich
Student ID: SJEPRo2412

Educational research prototype only. Not for clinical diagnosis.
Install:
    pip install pandas numpy scikit-learn joblib
Run:
    python diabetes_model.py
"""
import pandas as pd
import numpy as np
import joblib

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score
)

DATA_URL = (
    "https://raw.githubusercontent.com/jbrownlee/Datasets/"
    "master/pima-indians-diabetes.data.csv"
)

COLUMNS = [
    "Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
    "Insulin", "BMI", "DiabetesPedigreeFunction", "Age", "Outcome"
]
FEATURES = COLUMNS[:-1]
ZERO_AS_MISSING = [
    "Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"
]

def load_data():
    df = pd.read_csv(DATA_URL, header=None, names=COLUMNS)
    print("Dataset shape:", df.shape)
    print("\nClass distribution:\n", df["Outcome"].value_counts())
    print("\nMissing values before cleaning:\n", df.isna().sum())
    print("Duplicate rows:", df.duplicated().sum())
    df = df.drop_duplicates().copy()

    # In this dataset, zero is treated as an invalid/missing measurement
    # for these clinical fields. Pregnancies and target are not changed.
    for col in ZERO_AS_MISSING:
        df.loc[df[col] == 0, col] = np.nan

    return df
```

```
def build_models():
    # Imputation is inside each pipeline to prevent leakage.
    logistic = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(
            max_iter=2000, class_weight="balanced", random_state=42
        ))
    ])

    tree = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("classifier", DecisionTreeClassifier(
            max_depth=4, min_samples_leaf=8,
            class_weight="balanced", random_state=42
        ))
    ])

    forest = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("classifier", RandomForestClassifier(
            n_estimators=400, min_samples_leaf=3,
            class_weight="balanced", random_state=42,
            n_jobs=-1
        ))
    ])

    return {
        "Logistic Regression": logistic,
        "Decision Tree": tree,
        "Random Forest": forest
    }

def main():
    df = load_data()
    X = df[FEATURES]
    y = df["Outcome"].astype(int)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    models = build_models()
    fitted = {}
    rows = []

    for name, model in models.items():
        cv_f1 = cross_val_score(
            model, X_train, y_train, cv=cv, scoring="f1"
        )
        model.fit(X_train, y_train)
        predictions = model.predict(X_test)

        probabilities = None
        if hasattr(model, "predict_proba"):
            probabilities = model.predict_proba(X_test)[:, 1]

        metrics = {
            "Model": name,
            "CV F1 mean": cv_f1.mean(),
            "Accuracy": accuracy_score(y_test, predictions),
            "Precision": precision_score(
                y_test, predictions, zero_division=0
            ),
            "Recall": recall_score(
                y_test, predictions, zero_division=0
            ),
            "F1-score": f1_score(
                y_test, predictions, zero_division=0
            ),
            "ROC-AUC": (
                roc_auc_score(y_test, probabilities)
                if probabilities is not None else np.nan
            )
        }
        rows.append(metrics)
        fitted[name] = model

        print("\\n" + "=" * 65)
        print(name)
        print("Metrics:", metrics)
        print("Confusion matrix:\\n", confusion_matrix(y_test, predictions))
        print("Classification report:\\n",
              classification_report(y_test, predictions, zero_division=0))

    results = pd.DataFrame(rows).sort_values(
        by="F1-score", ascending=False
```

```
    )
    print("\\nMODEL COMPARISON\\n", results.to_string(index=False))

    best_name = results.iloc[0]["Model"]
    best_model = fitted[best_name]
    joblib.dump(best_model, "diabetes_risk_model.joblib")
    print("\\nSaved best model:", best_name,
          "-> diabetes_risk_model.joblib")

if __name__ == "__main__":
    main()
```

APPENDIX B: SAMPLE PREDICTION INPUT

After running Appendix A and creating the saved model file, the following optional snippet can be used in a separate Python file to generate a prediction for a single example record. The example values are illustrative only and must not be interpreted as a real person's clinical measurements.

```
import joblib
import pandas as pd

model = joblib.load("diabetes_risk_model.joblib")

example = pd.DataFrame([{
    "Pregnancies": 2,
    "Glucose": 120,
    "BloodPressure": 70,
    "SkinThickness": 25,
    "Insulin": 80,
    "BMI": 28.5,
    "DiabetesPedigreeFunction": 0.45,
    "Age": 35
}])

prediction = model.predict(example)[0]
probability = model.predict_proba(example)[0, 1]

print("Predicted class:", "Positive" if prediction == 1 else "Negative")
print("Estimated model probability:", round(probability, 3))
print("For educational demonstration only; not a medical diagnosis.")
```